Исследование применимости архитектур GNN к задачам дискретной оптимизации на примере задачи Graph 3-coloring

Патрикеева Елизавета Владимировна*

patrikeeva.ev@phystech.edu

Дмитриев Артем Андреевич†

dmitriev.artem@phystech.edu

Ярмошик Демьян Валерьевич‡

yarmoshik.dv@phystech.edu

Кафедра дискретной математики

Московский физико-технический институт

15 мая 2026 г.

Работа выполнена в рамках учебного проекта по курсу «Математический практикум» в течение весеннего семестра 2026 учебного года.

*Студентка 2 курса, ФПМИ, МФТИ

†Студент 2 курса, ФПМИ, МФТИ

‡Научный руководитель, к.ф.-м.н., асс

Аннотация

Задачи комбинаторной (дискретной) оптимизации на графах лежат в основе множества современных прикладных и вычислительных проблем, варьирующихся от маршрутизации и распределения радиочастот до анализа сложных биологических и социальных сетей. Одной из фундаментальных и наиболее изученных математических моделей в этой области является задача о раскраске графа. В частности, задача взвешенной 3-раскраски (Weighted Graph 3-Coloring), рассматриваемая в данной работе, заключается в таком назначении одного из трех состояний (цветов) каждой вершине графа, при котором минимизируется суммарный вес «конфликтных» (монохроматических) ребер, соединяющих вершины одинакового цвета.

С точки зрения теории вычислительной сложности, классическая задача определения возможности раскраски графа ровно в три цвета без конфликтов является NP-полной.Как следствие, ее оптимизационная постановка — минимизация штрафа за монохроматические ребра — относится к классу NP-трудных проблем. Это означает, что применение точных алгоритмов, таких как целочисленное линейное программирование (ILP) или полуопределенное программирование (SDP), сопряжено с проблемой комбинаторного взрыва. Для графов большой размерности (порядка десятков тысяч вершин) точные методы становятся вычислительно неэффективными или физически нереализуемыми. Существующие классические эвристики работают быстрее, однако часто застревают в локальных оптимумах и обладают ограниченной масштабируемостью.

В связи с этим в последние годы наблюдается стремительный рост исследовательского интереса к применению методов глубокого обучения (Deep Learning), а именно — графовых нейронных сетей (Graph Neural Networks, GNN), для приближенного решения NP-трудных задач. Актуальность применения GNN обусловлена их архитектурной спецификой: они способны эффективно агрегировать топологическую информацию и работать с графами произвольного размера.

Цель

Целью данной задачи является разработка эффективного алгоритма 3-раскраски для минимизации суммы взвешенных монохроматических рёбер.

Предпосылки

В общем случае большинство приближённых алгоритмов раскраски можно разделить на:

- **Эвристики:** работают быстро, дают локальный оптимум без оценки нижней границы.
- **Методы математического программирования (ILP, IQP, SDP):** работают медленно, могут оценить нижнюю границу, сходятся к глобальному оптимуму.

Поскольку универсального решения не существует, мы сосредоточимся на следующем классе графов:

- 20 000 вершин равномерно сгенерированы внутри квадрата.
- Каждая вершина случайным образом соединена с некоторым подмножеством из 60 ближайших соседей (по евклидову расстоянию).
- Веса рёбер распределены логнормально.

Содержание

1. **Теория графов и постановка задачи**

   Графы, матрицы, Лапласиан, раскраска, NP-трудность, Max 3-Cut

2. **Математическое программирование и классические эвристики**

   ILP, LP/SDP-релаксации, спектральная кластеризация, Kernighan–Lin, SA

3. **Методы машинного обучения для задачи раскраски**

   GNN, дифференцируемая оптимизация, DiffILO, обучение с подкреплением

4. **Вычислительная стратегия и экспериментальный план**

   Архитектура решения, параметры, метрики, нижняя граница

Глава 1. Теория графов и постановка задачи

1.1. Основные определения

Определение 1.1 (Граф)

Граф — пара $G = (V, E)$, где $V = \{1, \dots, n\}$ — конечное множество вершин, $E \subseteq \binom{V}{2}$ — множество неупорядоченных пар (рёбер). Для неориентированного графа $(u, v) \in E \Leftrightarrow (v, u) \in E$.

Определение 1.2 (Взвешенный граф)

Взвешенный граф $G = (V, E, w)$ дополнительно наделён весовой функцией $w : E \to \mathbb{R}^+$. В рассматриваемом классе задач веса имеют логнормальное распределение: $w_{ij} = e^{\xi_{ij}}$, где $\xi_{ij} \sim \mathcal{N}(0, 1)$. Это означает, что медианный вес равен 1, а среднее значение $\mathbb{E}[w] = e^{1/2} \approx 1{,}65$, при этом распределение имеет тяжёлый правый хвост — отдельные рёбра могут иметь значительно больший вес.

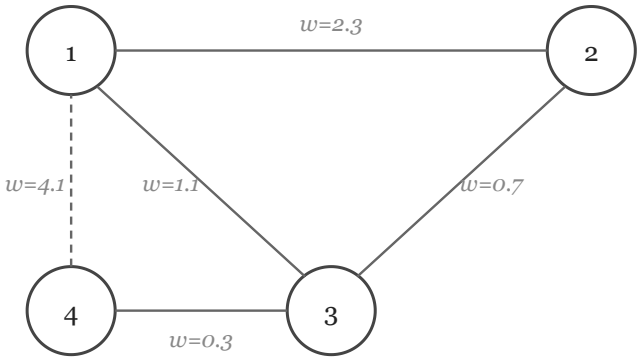


Рис. 1.1. Пример взвешенного графа с 4 вершинами и 5 рёбрами. Пунктирное ребро (1,4) имеет наибольший вес.

1.2. Матричное представление

Определение 1.3 (Матрица смежности)

Матрица смежности $A \in \mathbb{R}^{n \times n}$ взвешенного графа:

$$A_{ij} = \begin{cases} w_{ij}, & (i,j) \in E, \\ 0, & \text{иначе}. \end{cases}$$

Для неориентированного графа A симметрична: $A = A^{\top}$.

Для графа на рис. 1.1:

$$A = \begin{pmatrix} 0 & 2.3 & 1.1 & 4.1 \\ 2.3 & 0 & 0.7 & 0 \\ 1.1 & 0.7 & 0 & 0.3 \\ 4.1 & 0 & 0.3 & 0 \end{pmatrix}$$

Определение 1.4 (Матрица степеней и Лапласиан)

Взвешенная степень вершины i: $d_i = \sum_{j:(i,j)\in E} w_{ij}$. Матрица степеней $D = \operatorname{diag}(d_1, \ldots, d_n)$. **Лапласиан**:

$$L = D - A.$$

Нормализованный Лапласиан: $L_{\text{sym}} = D^{-1/2} L\, D^{-1/2} = I - D^{-1/2} A\, D^{-1/2}$.

Свойство 1.1 (Квадратичная форма Лапласиана)

Для любого вектора $x \in \mathbb{R}^n$:

$$x^{\top} L x = \sum_{(i,j)\in E} w_{ij} (x_i - x_j)^2.$$

Отсюда $L \succeq 0$ (положительно полуопределена). Собственные значения $0 = \lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$. Второе собственное значение $\lambda_2 > 0$ тогда и только тогда, когда граф связен. Величина λ_2 называется алгебраической связностью (Fiedler, 1973).

Квадратичная форма $x^{\top} L x$ измеряет «гладкость» вектора x на графе: она мала, когда соседние вершины имеют близкие значения. Собственные векторы Лапласиана можно рассматривать как обобщение преобразования Фурье на графы — это основа спектральных методов, которые будут использованы далее.

1.3. Задача раскраски графа

> **Определение 1.5 (k-раскраска)**
>
> k-раскраска графа G — функция $c : V \to [k] = \{1, \dots, k\}$, назначающая каждой вершине цвет. Раскраска *правильная*, если $\forall (u, v) \in E : c(u) \neq c(v)$. Хроматическое число $\chi(G)$ — минимальное k, при котором существует правильная раскраска.

Определение $\chi(G)$ — NP-трудная задача. Даже проверка 3-раскрашиваемости NP-полна (Karp, 1972). Для произвольных плотных графов правильная 3-раскраска может не существовать. В таких случаях естественно перейти к задаче минимизации нарушений.

1.4. Формулировка задачи минимизации

> **Задача (Weighted Min-Violation 3-Coloring)**
>
> Дан $G = (V, E, w)$. Найти $c^* : V \to \{1, 2, 3\}$, минимизирующую
>
> $$f(c) = \sum_{(u,v) \in E} w_{uv} \cdot \mathbb{I}[c(u) = c(v)] \;\to\; \min.$$
>
> Ребро (u, v) с $c(u) = c(v)$ называется *монохроматическим* (нарушением).

Тривиальная нижняя граница: $f(c) \geq 0$, достигаемая только при существовании правильной 3-раскраски. Тривиальная верхняя граница (случайная раскраска): при $c(v) \sim \text{Uniform}\{1, 2, 3\}$ ожидаемое значение $\mathbb{E}[f] = W/3$, где $W = \sum_{e \in E} w_e$ — полный вес.

Числовой пример

Рассмотрим граф на рис. 1.1 с рёбрами $\{(1,2), (1,3), (1,4), (2,3), (3,4)\}$ и весами $\{2.3, 1.1, 4.1, 0.7, 0.3\}$. Полный вес $W = 8.5$.

Раскраска c	**Монохроматические рёбра**	$f(c)$	**Доля от** W
$c = (1, 2, 3, 2)$	нет	0	0%
$c = (1, 1, 2, 3)$	(1,2): $w = 2.3$	2.3	27%
$c = (1, 2, 1, 3)$	(1,3): $w = 1.1$	1.1	13%
$c = (1, 2, 2, 2)$	(2,3): 0.7, (3,4): 0.3	1.0	12%
Случайная (ожид.)	—	$W/3 \approx 2.83$	33%

В данном примере существует правильная раскраска $c = (1, 2, 3, 2)$ с $f = 0$. Однако для графов с высокой плотностью рёбер правильная 3-раскраска может не существовать — хроматическое число может быть больше 3. В этом случае любая раскраска неизбежно содержит нарушения, и задача состоит в минимизации их суммарного веса.

Заметим, что вклад ребра в целевую функцию пропорционален его весу. Ребро (1,4) с весом 4.1 вносит наибольший вклад при нарушении — алгоритм должен в первую очередь обеспечить разные цвета у вершин 1 и 4. Это наблюдение мотивирует жадные стратегии, учитывающие веса рёбер.

1.5. Эквивалентность с Max k-Cut

Утверждение 1.1

Минимизация $f(c)$ эквивалентна задаче Max 3-Cut:

$$f(c) = W - \underbrace{\sum_{(u,v) \in E} w_{uv} \cdot \mathbb{I}[c(u) \neq c(v)]}_{\mathrm{Cut}(c)}.$$

Таким образом, $\min f(c) = W - \max \mathrm{Cut}(c)$.

Это наблюдение фундаментально: для Max k-Cut известны SDP-релаксации с доказанными гарантиями аппроксимации (Frieze, Jerrum, 1997). Для $k = 3$ соответствующий алгоритм гарантирует решение не хуже $12/13 \approx 0{,}923$ от оптимума при равномерных весах, и $\approx 0{,}836$ в общем случае.

1.6. Класс рассматриваемых графов

Графы генерируются по следующему протоколу:

1. $n = 20\,000$ вершин размещаются равномерно в $[0, 1]^2$.
2. Для каждой вершины определяются $K = 60$ ближайших соседей.
3. Ребро от i к соседу j создаётся с вероятностью $p_i = (i \bmod 6)/10$.
4. Вес ребра: $w_{ij} = e^{\xi}$, $\xi \sim \mathcal{N}(0, 1)$.
5. Матрица симметризуется: $A \leftarrow A + A^{\top}$ (веса суммируются).

Неоднородность связности — ключевая особенность. Вероятности $p \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$ порождают 6 классов вершин с существенно различной средней степенью. Вершины с $i \bmod 6 = 0$ не инициируют исходящих рёбер, хотя могут получать входящие. После симметризации ожидаемое число рёбер $\approx$ 150--300 тыс., что делает граф разреженным: $|E| / \binom{n}{2} < 0.002$.

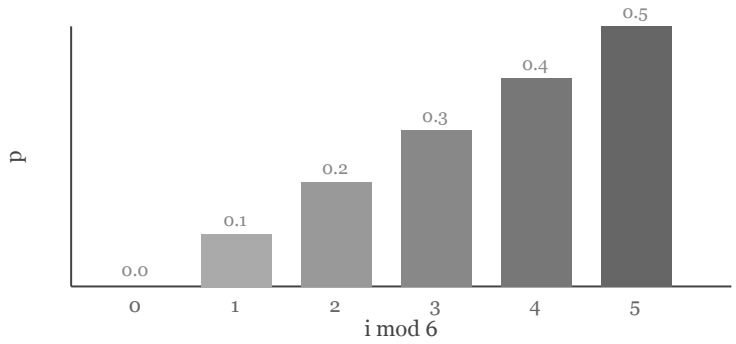


Рис. 1.2. Вероятность связи p_i в зависимости от класса вершины $i \bmod 6$.

Глава 2. Математическое программирование и классические эвристики

2.1. Формулировка как ILP

Формулировка 2.1 (ILP для Min-Violation 3-Coloring)

Бинарные переменные: $x_{ik} \in \{0,1\}$ — вершина i окрашена в цвет k; $y_{ij} \in \{0,1\}$ — ребро монохроматическое.

$$\min \sum_{(i,j)\in E} w_{ij}\, y_{ij}$$

$$\sum_{k=1}^{3} x_{ik} = 1 \quad \forall i \in V \qquad \text{(единственность цвета)}$$

$$y_{ij} \geq x_{ik} + x_{jk} - 1 \quad \forall (i,j) \in E,\ k \in \{1,2,3\} \qquad \text{(детекция нарушений)}$$

Размерность задачи для нашего графа: $3n + |E| \approx 260\,000$ переменных, $n + 3|E| \approx 620\,000$ ограничений. При отсутствии коммерческих солверов (Gurobi, CPLEX) прямое решение за 30 минут невыполнимо, однако формулировка полезна для построения релаксаций.

2.2. LP-релаксация и нижняя граница

Замена $x_{ik} \in \{0,1\}$ на $x_{ik} \in [0,1]$ и $y_{ij} \in [0,1]$ даёт задачу линейного программирования. Оптимум LP-релаксации z^*_{LP} является нижней границей:

$$z^*_{LP} \leq z^*_{ILP} = f(c^*).$$

Интуиция: LP допускает «дробные» назначения, например, $x_{i1} = x_{i2} = x_{i3} = 1/3$. При таком назначении y_{ij} может быть сколь угодно малым, что занижает целевую функцию по сравнению с любым дискретным решением.

Определение 2.1 (Зазор оптимальности)

Пусть f^* — значение, найденное эвристикой, LB — нижняя граница. Зазор оптимальности:

$$\text{gap} = \frac{f^* - \text{LB}}{f^*} \times 100\%.$$

Зазор показывает, на сколько процентов наше решение может отличаться от глобального оптимума. Gap = 0% означает доказанную оптимальность.

2.3. SDP-релаксация для Max 3-Cut

Воспользуемся эквивалентностью с Max 3-Cut. Каждой вершине i сопоставим единичный вектор $v_i \in \mathbb{R}^d$. Наблюдение: если v_i, v_j выбраны из множества $\{e_1, e_2, e_3\}$ (стандартный базис), то $v_i^\top v_j = \mathbb{I}[c(i) = c(j)]$, и значение разреза:

$$\text{Cut} = \sum_{(i,j)\in E} w_{ij}(1 - v_i^\top v_j) \cdot \frac{k-1}{k}.$$

Формулировка 2.2 (SDP-релаксация, Frieze–Jerrum)

Обозначим $X_{ij} = v_i^\top v_j$ (матрица Грама). Задача Max 3-Cut релаксируется в SDP:

$$\max \frac{1}{3} \sum_{(i,j)\in E} w_{ij}(1 - X_{ij})$$

$$\text{при } X_{ii} = 1 \;\; \forall i, \quad X_{ij} \geq -\tfrac{1}{2} \;\; \forall i \neq j, \quad X \succeq 0.$$

Оптимум SDP даёт верхнюю границу Max 3-Cut и, следовательно, нижнюю границу нашей задачи: $\text{LB} = W - \text{SDP}_{\text{opt}}$.

Матрица $X \in \mathbb{R}^{n\times n}$ при $n = 20\,000$ содержит 2×10^8 элементов. Прямое решение SDP возможно, но требует значительных ресурсов. Практические альтернативы:

2.3.1. Спектральная оценка (быстрая)

Верхняя граница Max k-Cut связана с наибольшим собственным значением Лапласиана. Для взвешенного графа:

$$\mathrm{Cut}_{\max} \le \frac{k-1}{2k} \cdot n \cdot \lambda_{\max}\left(\frac{L}{n}\right) = \frac{k-1}{2k} \sum_{(i,j)\in E} w_{ij}\left(1 + \frac{1}{k-1}\right).$$

Вычисление $\lambda_{\max}(L)$ для разреженной матрицы через метод Ланцоша занимает $O(|E| \cdot t)$ операций, где $t \sim$ 50--100 — число итераций. На нашем графе это секунды.

2.4. Спектральная кластеризация

Спектральная кластеризация — один из наиболее эффективных методов инициализации разбиения графа. Она использует информацию о глобальной структуре, закодированную в собственных векторах Лапласиана.

Алгоритм 2.1 (Спектральная 3-раскраска)

Вход: граф $G = (V, E, w)$. **Выход:** раскраска $c : V \to \{1, 2, 3\}$.

1. Построить $L_{\mathrm{sym}} = I - D^{-1/2} A D^{-1/2}$.
2. Найти собственные векторы u_2, u_3, отвечающие λ_2, λ_3.
3. Сформировать $U = [u_2 \mid u_3] \in \mathbb{R}^{n \times 2}$.
4. Нормализовать строки: $\tilde{u}_i = u_i / \|u_i\|$.
5. Применить k-means ($k = 3$) к строкам $\tilde{U}$.
6. Назначить $c(i)$ по номеру кластера.

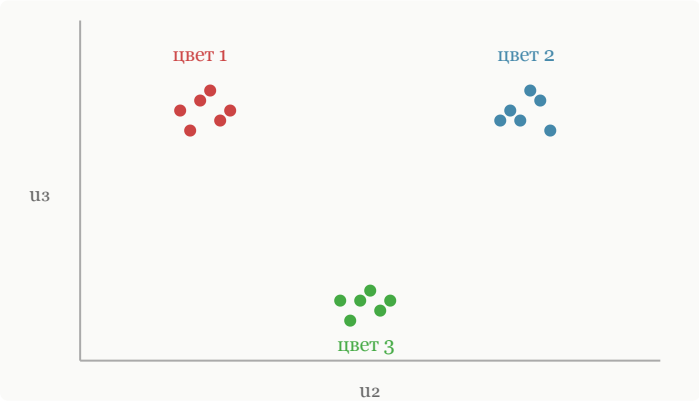


Рис. 2.1. Спектральное вложение вершин в пространство (u_2, u_3). Кластеры соответствуют цветовым классам.

Обоснование: минимизация $\sum w_{ij}(x_i - x_j)^2$ при $\|x\| = 1$ и $x \perp \mathbf{1}$ даёт вектор Фидлера u_2. Для k-way разбиения используются $k-1$ собственных векторов — они обеспечивают представление, в котором вершины одного кластера оказываются близки.

2.5. Kernighan–Lin

После спектральной инициализации применяется адаптация алгоритма Kernighan–Lin для k-way разбиения. В отличие от жадного подхода (greedy), KL рассматривает *последовательности* ходов, что позволяет преодолевать локальные минимумы.

Алгоритм 2.2 (K-way Kernighan–Lin)

1. **Инициализация:** для каждой вершины v вычислить gain перекраски в лучший альтернативный цвет: $g(v) = \max_{k' \neq c(v)} \left[\sum_{u \in \mathcal{N}(v)} w_{vu} (\mathbb{I}[c(v) = c(u)] - \mathbb{I}[k' = c(u)]) \right]$.
2. **Проход:** выбрать v^* с максимальным $g(v^*)$, перекрасить, заблокировать. Повторить n раз.
3. **Откат:** из последовательности n ходов выбрать префикс длины m^*, максимизирующий суммарное улучшение $\sum_{t=1}^{m} g_t$.
4. Применить первые m^* ходов, разблокировать вершины, перейти к шагу 1.

Остановка: когда $m^* = 0$ (нет улучшающего префикса).

2.6. Simulated Annealing

SA дополняет KL возможностью выхода из локальных минимумов путём стохастического принятия ухудшений.

Алгоритм 2.3 (SA для 3-раскраски)

Параметры: T_0 — начальная температура, α — коэффициент охлаждения, N_{iter} — число итераций.

1. Инициализация: $c \leftarrow c_{\text{KL}}$, $T \leftarrow T_0$, $f_{\text{best}} \leftarrow f(c)$.
2. Для $t = 1, \ldots, N_{\text{iter}}$:

   a) Выбрать случайную вершину v, случайный $k' \neq c(v)$.

   b) $\Delta f = \sum_{u \in \mathcal{N}(v)} w_{vu} \big(\mathbb{I}[k' = c(u)] - \mathbb{I}[c(v) = c(u)] \big)$.

   c) Если $\Delta f < 0$ или $\text{rand}() < \exp(-\Delta f / T)$: принять.

   d) $T \leftarrow \alpha \cdot T$.

   e) Если $f < f_{\text{best}}$: сохранить лучшее.

Выбор параметров: T_0 устанавливается так, чтобы начальная вероятность принятия ухудшений составляла ~40–60%. Для этого выполняется короткий пробный прогон с подсчётом среднего $|\Delta f|$. Коэффициент $\alpha = 1 - 5/N_{\text{iter}}$ обеспечивает достижение финальной температуры $T_f \approx T_0 \cdot e^{-5}$ к концу прогона.

Вычислительная эффективность: при средней степени вершины $\bar{d} \approx 30$ каждый ход требует ~30 операций. За 900 секунд на современном CPU выполнимо порядка 10^7--10^8 итераций.

Глава 3. Методы машинного обучения для задачи раскраски

3.1. Мотивация применения ML

Классические эвристики (SA, KL) не используют знание о распределении входных данных. Если известно, что графы порождаются одним генератором, можно обучить модель, которая извлекает структурные закономерности из обучающей выборки и переносит их на новые экземпляры. Это позволяет получить качественное начальное приближение за время одного прямого прохода нейросети, существенно сокращая фазу локального поиска.

3.2. Графовые нейронные сети (GNN)

> **Определение 3.1 (Message Passing Neural Network)**
>
> GNN — нейросеть, работающая с графовыми данными посредством итеративной передачи сообщений:
>
> $$m_v^{(\ell)} = \bigoplus_{u \in \mathcal{N}(v)} \psi^{(\ell)}\left(h_u^{(\ell)}, w_{vu}\right), \qquad h_v^{(\ell+1)} = \phi^{(\ell)}\left(h_v^{(\ell)},\ m_v^{(\ell)}\right),$$
>
> где $h_v^{(\ell)} \in \mathbb{R}^d$ — представление вершины на слое ℓ, $\bigoplus$ — агрегация (сумма или среднее), ψ, ϕ — обучаемые функции (MLP). После L слоёв каждая вершина аккумулирует информацию из L-окрестности.

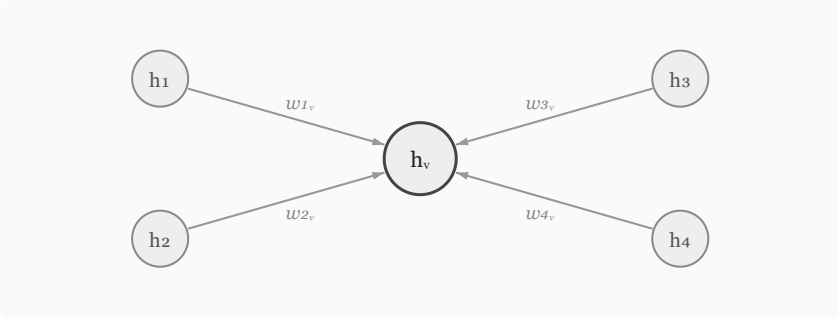


Рис. 3.1. Один шаг передачи сообщений: вершина v агрегирует представления соседей.

Два распространённых варианта GNN:

GCN (Kipf, Welling, 2017): $H^{(\ell+1)} = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(\ell)}W^{(\ell)})$, где $\tilde{A} = A + I$. Простая архитектура, но использует все рёбра при каждом проходе.

GraphSAGE (Hamilton et al., 2017): применяет сэмплирование фиксированного числа соседей на каждом слое. Масштабируется на графы с миллионами вершин без необходимости хранить всю матрицу смежности. Рекомендуется для нашего графа с 20 000 вершинами.

3.3. Обучение без учителя через дифференцируемую целевую функцию

Центральная идея: использовать целевую функцию задачи оптимизации *непосредственно* как функцию потерь нейросети. Для этого необходимо сделать дискретную задачу дифференцируемой.

3.3.1. Непрерывная релаксация

> **Определение 3.2 (Мягкое назначение через softmax)**
>
> Пусть GNN с параметрами θ выдаёт логиты $z_v \in \mathbb{R}^3$ для каждой вершины. Мягкое назначение:
>
> $$p_{v,k} = \frac{\exp(z_{v,k}/\tau)}{\sum_{k'=1}^{3}\exp(z_{v,k'}/\tau)}, \qquad p_v = (p_{v,1}, p_{v,2}, p_{v,3}) \in \Delta^2,$$
>
> где $\tau > 0$ — температурный параметр, Δ^2 — вероятностный симплекс. При $\tau \to 0$ распределение вырождается в $\arg\max$, при $\tau \to \infty$ — в равномерное.

3.3.2. Функция потерь

Вероятность монохроматичности ребра (u, v) при независимом сэмплировании цветов из p_u и p_v:

$$P_{\text{mono}}(u, v) = \sum_{k=1}^{3} p_{u,k} \cdot p_{v,k} = p_u^\top p_v.$$

Формулировка 3.1 (Функция потерь)

$$\mathcal{L}(\theta) = \sum_{(u,v)\in E} w_{uv}\, p_u^\top p_v \;-\; \lambda \sum_{v\in V} H(p_v),$$

где $H(p) = -\sum_k p_k \ln p_k$ — энтропия. Первый член минимизирует ожидаемый вес нарушений, второй — регуляризатор, поощряющий уверенные назначения (низкую энтропию) при малых λ. Коэффициент λ убывает в процессе обучения, обеспечивая переход от исследования к эксплуатации.

Минимум $p_u^\top p_v$ при фиксированных $p_u \in \Delta^2$ достигается при ортогональных one-hot векторах: $p_u^\top p_v = 0$, что соответствует разным цветам. Максимум — при $p_u = p_v$, тогда $p_u^\top p_v = \sum p_{u,k}^2 \le 1$. Таким образом, минимизация $\mathcal{L}$ поощряет различные цвета у соседних вершин, особенно на тяжёлых рёбрах.

3.3.3. Отжиг температуры

Расписание температуры $\tau(t)$ критически важно для сходимости. Начальное $\tau = 1$ обеспечивает гладкий ландшафт потерь. Постепенное снижение до $\tau = 0.05$ заостряет распределения p_v, приближая их к дискретным назначениям. Типичное расписание:

$$\tau(t) = \tau_{\min} + (\tau_{\max} - \tau_{\min}) \cdot \cos^2\!\left(\frac{\pi t}{2T}\right),$$

где T — полное число эпох.

3.4. Метод DiffILO

DiffILO (Geng et al., ICLR 2025) — общий метод решения задач целочисленного линейного программирования через градиентный спуск. В отличие от специализированных подходов, он применим к произвольным ILP без необходимости вручную конструировать дифференцируемый суррогат.

Формулировка 3.2 (Три этапа DiffILO)

Этап 1. Вероятностная релаксация: $x_i \in \{0,1\} \;\to\; x_i \sim \text{Bernoulli}(\hat{x}_i)$, $\hat{x}_i \in [0,1]$.

Этап 2. Штрафной метод:

$$\min_{\hat{x}\in[0,1]^n} F_\mu(\hat{x}) = c^\top \hat{x} + \mu \sum_{j=1}^{m} \mathbb{E}_{x\sim p(\cdot|\hat{x})}\big[\max(a_j^\top x - b_j,\ 0)\big].$$

При $\mu > \mu^*$ оптимальные $\hat{x}_i^*$ бинарны для всех i с $c_i \neq 0$.

Этап 3. Reparameterization trick для оценки градиентов: $\epsilon_i \sim U(0,1)$, непрерывный суррогат $\xi_i = \sigma\big((\text{logit}(\hat{x}_i) + \text{logit}(\epsilon_i))/\tau\big)$, бинарное $\psi_i = [\xi_i > 0.5]$. Для вычисления значения используется ψ, для градиентов — ξ.

Для нашей задачи переменные $\hat{x}_{ik}$ кодируют вероятности назначения цвета k вершине i. Ограничение $\sum_k x_{ik} = 1$ входит в штрафной член. Преимущество DiffILO — формальная гарантия корректности: при правильном μ непрерывный оптимум совпадает с дискретным.

3.5. Обучение с подкреплением

Определение 3.3 (MDP для раскраски)

Состояние s_t — частичная раскраска; действие $a_t = (v, k)$ — назначение цвета k вершине v; награда $r_t = -\Delta f$ — уменьшение суммы нарушений. Политика $\pi_\theta(a|s)$ параметризуется GNN.

Градиент политики (REINFORCE, Williams 1992):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} (R_t - b)\,\nabla_\theta \ln \pi_\theta(a_t|s_t)\right],$$

где $R_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$, b — базовая линия (скользящее среднее вознаграждений). Применение: обучение на графах с $n = 200$--2000, перенос на целевой граф с $n = 20\,000$ благодаря инвариантности GNN к числу вершин.

3.6. Архитектура для нашей задачи

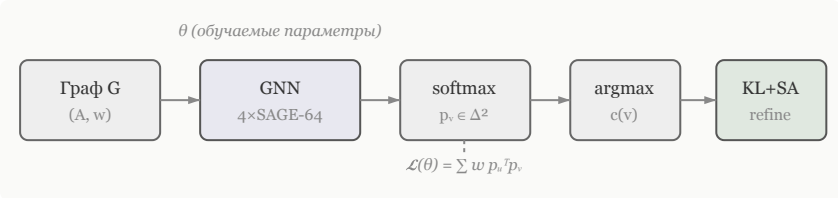


Рис. 3.2. Архитектура гибридного решения: GNN выдаёт мягкое назначение, после дискретизации — локальный поиск.

Обоснование выбора компонентов:

- **GraphSAGE** (а не GCN): сэмплирование соседей масштабируется лучше при $n = 20\,000$ и средней степени ~30.
- **4 слоя:** рецептивное поле охватывает 4-окрестность вершины, что достаточно для захвата локальной структуры kNN-графа.
- **hidden_dim = 64:** баланс между выразительностью и скоростью inference.
- **Unsupervised loss:** не требует размеченных данных — модель обучается на самой целевой функции.

3.6.1. Начальные признаки вершин

Каждая вершина v описывается вектором признаков $x_v \in \mathbb{R}^{d_0}$, подаваемым на вход GNN:

Признак	Размерность	Описание
Взвешенная степень d_v	1	$\sum_{u \in \mathcal{N}(v)} w_{vu}$
Число соседей $\lvert\mathcal{N}(v)\rvert$	1	Невзвешенная степень
Класс связности	1	$p_v = (v \bmod 6)/10$
Координаты	2	$(x_v, y_v) \in [0,1]^2$
Средний вес ребра	1	$d_v / \lvert\mathcal{N}(v)\rvert$

Итого $d_0 = 6$. Координаты вершин несут информацию о пространственной структуре kNN-графа, что помогает GNN обнаруживать геометрические кластеры.

3.6.2. Процедура обучения

Алгоритм 3.1 (Обучение GNN для 3-раскраски)

Вход: обучающие графы $\{G_1, \ldots, G_M\}$, число эпох T, расписания $\tau(t)$, $\lambda(t)$.

Выход: обученные параметры θ^*.

1. Инициализировать θ (Xavier initialization).
2. Для $t = 1, \ldots, T$:
   a) Вычислить τ_t и λ_t по расписанию.
   b) Для каждого графа G_m:
   — Forward pass: $z = \mathrm{GNN}_\theta(G_m)$, $p = \mathrm{softmax}(z/\tau_t)$.
   — Вычислить loss: $\mathcal{L} = \sum_{(u,v)\in E_m} w_{uv} p_u^\top p_v - \lambda_t \sum_v H(p_v)$.
   — Backward pass: $\nabla_\theta \mathcal{L}$.
   c) Обновить $\theta \leftarrow \theta - \eta_t \nabla_\theta \mathcal{L}$ (Adam).
   d) Каждые 10 эпох: оценить $\bar{f}$ на валидации (argmax + подсчёт нарушений).
   e) Сохранить θ^*, дающее минимальное $\bar{f}$ на валидации.

3.6.3. Анализ сходимости

На начальных эпохах (высокое τ) функция потерь гладкая, и оптимизатор быстро находит область хороших решений. По мере снижения τ распределения p_v заостряются, и ландшафт потерь становится более «рельефным» с выраженными локальными минимумами. Cosine annealing обеспечивает плавный переход, избегая резкого ухудшения сходимости.

Типичная динамика обучения:

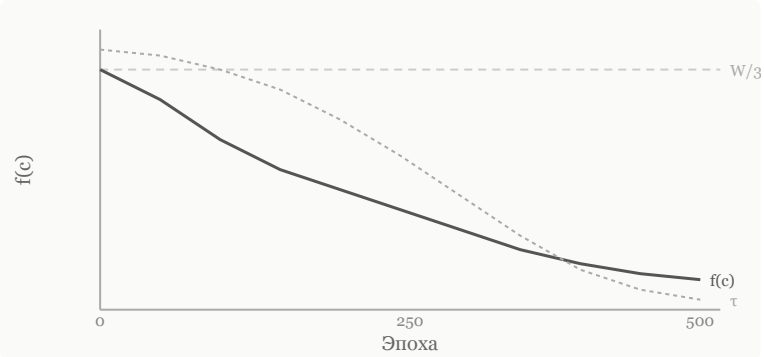


Рис. 3.3. Типичная динамика обучения. Сплошная линия — значение $f(c)$ после дискретизации на валидации. Пунктирная — температура τ. Горизонтальная линия — baseline (случайная раскраска).

3.6.4. Практические замечания

Распространённые проблемы при обучении и способы их решения:

Проблема	**Симптом**	**Решение**
Слишком высокая τ	Все $p_v \approx (1/3, 1/3, 1/3)$, loss не убывает	Начать с $\tau = 1$, не выше
Резкое снижение τ	Loss скачет, модель застревает	Cosine annealing вместо step decay
Малый μ в DiffILO	Ограничения не выполняются	Постепенно увеличивать μ
Большой learning rate	Осцилляции loss	lr = 10^{-3} с decay
Мало сэмплов (DiffILO)	Высокая дисперсия градиента	≥ 10 сэмплов на шаг

Глава 4. Вычислительная стратегия и экспериментальный план

4.1. Обзор архитектуры

Решение объединяет три парадигмы: машинное обучение (инициализация), классические эвристики (локальная оптимизация) и математическое программирование (нижняя граница). Каждый компонент решает свою подзадачу, компенсируя ограничения остальных.

Подзадача	Метод	Роль
Начальное приближение	GNN + спектральная кластеризация	Быстрая генерация качественной раскраски
Локальное улучшение	Kernighan–Lin, Simulated Annealing	Гарантированное монотонное улучшение (KL), выход из локальных минимумов (SA)
Оценка качества	Eigenvalue bound, LP-релаксация	Математически строгая нижняя граница

4.2. Фаза предобучения (офлайн)

Предобучение GNN выполняется до старта основного алгоритма и не входит в 30-минутный бюджет. Протокол:

1. Сгенерировать 50–100 графов тем же генератором с различными random seed.
2. Для каждого графа инициализировать GNN случайными весами.
3. Оптимизировать $\mathcal{L}(\theta)$ методом Adam (lr = 10^{-3}, cosine decay) в течение 500 эпох.
4. Расписание τ: косинусный отжиг $1.0 \to 0.05$. Расписание λ: $0.1 \to 0.001$.
5. На валидационной выборке (10 графов) выбрать эпоху с лучшим средним f после дискретизации.
6. Сохранить веса модели.

Обоснование: все графы порождаются одним генератором, поэтому GNN обучается на том же распределении и способна переносить найденные паттерны (например, учитывать неоднородность связности, вызванную структурой $p_i = (i \bmod 6)/10$).

4.3. Фаза решения (30 минут)

Время	Этап	Описание
0:00–0:30	Загрузка	Генерация или загрузка графа, построение sparse-структур
0:30–2:00	GNN inference	Forward pass, softmax, argmax — начальная раскраска c_1
2:00–4:00	Спектральная кластеризация	Lanczos для u_2, u_3, k-means — раскраска c_2
4:00–4:30	Выбор	$c_0 = \arg\min(f(c_1), f(c_2))$
4:30–10:00	Kernighan–Lin	Несколько раундов до сходимости
10:00–25:00	Simulated Annealing	Основной прогон с адаптивным T_0
25:00–28:00	Нижняя граница	Eigenvalue bound ($\lambda_{\max}$ через Lanczos)
28:00–30:00	Финализация	Gap, сохранение результата

4.4. Параметры алгоритмов

4.4.1. GNN

Параметр	Значение	Обоснование
Архитектура	4 слоя GraphSAGE	4-hop рецептивное поле покрывает локальную структуру
Hidden dim	64	Достаточно для 3-классовой задачи
Выход	Linear(64, 3) + softmax	3 цвета
Начальные признаки	d_v, p_v, координаты (x_v, y_v)	Степень + геометрия

4.4.2. Simulated Annealing

Параметр	Значение	Обоснование
T_0	$\overline{\lvert\Delta f\rvert}$ (средний абс. Δf при 1000 случайных ходах)	Начальная вероятность принятия ~50%
α	$1 - 5/N_{\text{iter}}$	Финальная $T \approx T_0 e^{-5}$
N_{iter}	$\sim 10^7$	15 минут при ~10^4 iter/sec

4.5. Нижняя граница

Eigenvalue bound: вычисляем $\lambda_{\max}(L)$ через `scipy.sparse.linalg.eigsh` (метод Ланцоша, $O(|E| \cdot t)$). Верхняя оценка Max 3-Cut:

$$\text{Cut}_{\max} \leq \frac{k-1}{k} \cdot \frac{n\lambda_{\max}}{2(k-1)} \cdot k = \frac{n\lambda_{\max}}{2}.$$

Нижняя граница нашей задачи: $\text{LB} = W - \text{Cut}_{\max}$. При наличии дополнительного времени можно решить LP-релаксацию формулировки 2.1 для более тесной границы.

4.6. Метрики

Метрика	Формула	Целевое значение
Целевая функция	$f(c) = \sum w_{uv}\,\mathbb{I}[c(u) = c(v)]$	Минимизировать
Нижняя граница	LB	Максимизировать
Gap	$(f - \text{LB})/f \times 100\%$	< 10%
Доля нарушений	$\lvert\{e : \text{mono}\}\rvert/\lvert E\rvert$	Минимизировать
Время	wall clock	$\leq$ 1800 с

4.7. Заключение

Предложенный подход сочетает математическое программирование, классические эвристики и машинное обучение. GNN обеспечивает начальное приближение, использующее распределение входных данных. Kernighan–Lin и Simulated Annealing последовательно улучшают его, а спектральная оценка и LP-релаксация дают математически строгую оценку зазора оптимальности.

Ключевые преимущества гибридного подхода: (1) ML-инициализация сокращает фазу сходимости локального поиска; (2) классические эвристики гарантируют качество, даже если GNN не обучилась идеально; (3) нижняя граница объективно оценивает результат без знания глобального оптимума.

Литература

1. Frieze, A., Jerrum, M. (1997). Improved approximation algorithms for MAX k-CUT and MAX BISECTION. *Algorithmica*, 18(1), 67–81.
2. Kipf, T.N., Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *Proc. ICLR*.
3. Hamilton, W.L., Ying, R., Leskovec, J. (2017). Inductive representation learning on large graphs. *Proc. NeurIPS*.
4. Geng, Z. et al. (2025). DiffILO: Differentiable Integer Linear Optimization. *Proc. ICLR*.
5. Kernighan, B.W., Lin, S. (1970). An efficient heuristic procedure for partitioning graphs. *Bell System Technical Journal*, 49(2), 291–307.
6. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
7. Karalias, N., Loukas, A. (2020). Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. *Proc. NeurIPS*.
8. Fiedler, M. (1973). Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2), 298–305.
9. Goemans, M.X., Williamson, D.P. (1995). Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *JACM*, 42(6), 1115–1145.
10. Williams, R.J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8, 229–256.